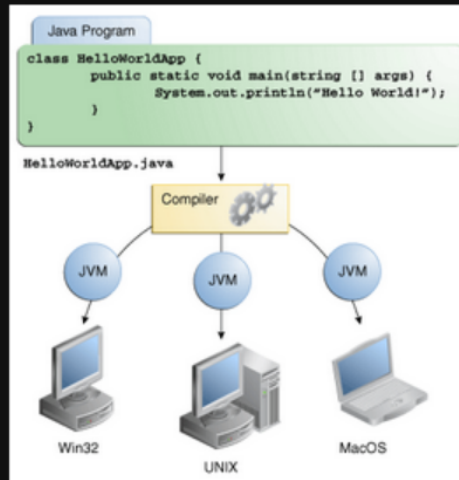


Java Programming Language

Created by <Academia de Código_>

Java Programming Environment

"Write Once, Run Anywhere"



Java is a Compiled/Interpreted language



Java is compiled into bytecode which is interpreted by the Java Virtual Machine (JVM).

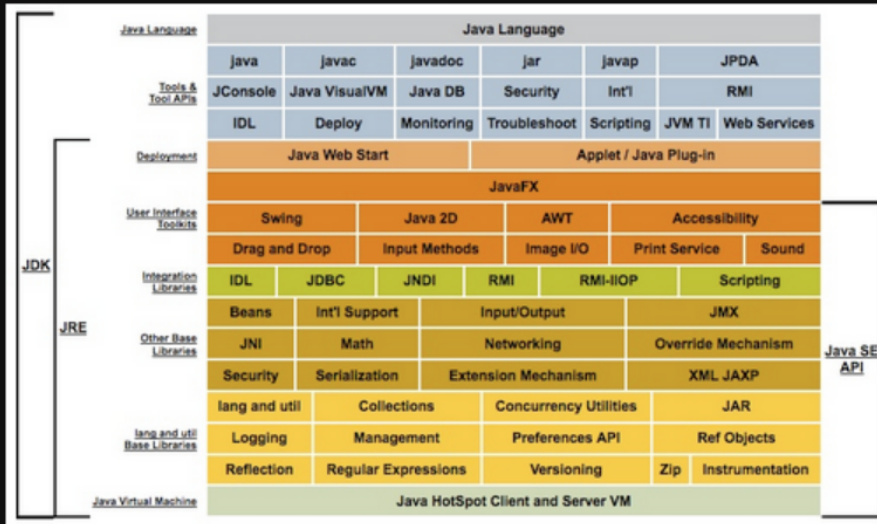
JVM languages

Many computer programming languages are used to produce computer software that runs on the JVM

- Clojure - a functional programming language
- Groovy - a dynamic programming and scripting language
- Scala - a statically-typed object-oriented and functional programming language
- Kotlin - a statically-typed language from JetBrains, the developers of IntelliJ IDEA
- JRuby - an implementation of Ruby
- Jython - an implementation of Python

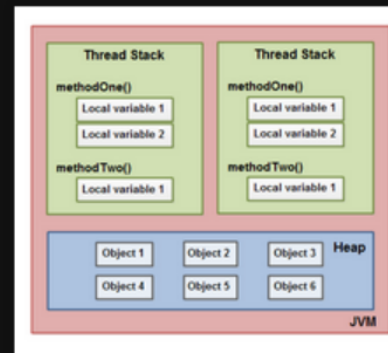
...and many more...

The Java Platform



The Java Memory Model

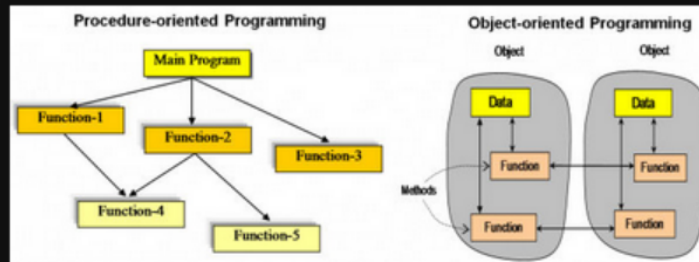
- Each thread has its own stack
- Local method variables reside on the stack
- Objects reside on the heap
- Just like a function, a Method is a collection of statements that are grouped together to perform an operation
- An Object is a combination of variables and methods



Object-Oriented Language



Programming Paradigms



{ Hands On }

Let's Hello World! in JAVA.

```
class HelloWorld {  
    public static void main(String[] args) {  
        System.out.println("Hello World!!");  
    }  
}
```

Introduction to Java

Java is a statically typed language

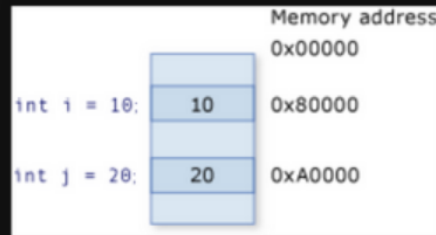
variables and expressions have a type that is known at compile time

Variables

- Primitive Types;
- Reference Types.

Primitive Types

- Built into the Java language and named by a reserved keyword
- Do not have any functionality other than simple data storage

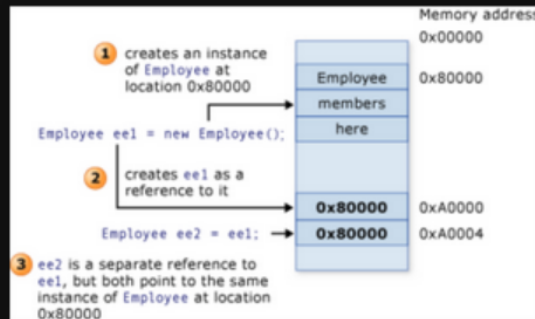


Primitive Types

Types	Default Size	Width	Default value	Range
boolean	1 bit		false	0 and 1
char	2 byte	16	'\u0000'	'\u0000' (or 0) to '\uffff' (or 65,535 inclusive)
byte	1 byte	8	0	-128 to 127
short	2 byte	16	0	-32,768 to 32,767
int	4 byte	32	0	-2,147,483,648 to 2,147,483,647(-2 ³¹ to -2 ³¹ -1)
long	8 byte	64	0L	-2 ⁶³ to -2 ⁶³ -1
float	4 byte	32	0.0f	1.4e -045 to 3.4e +038
double	8 byte	64	0.0d	4.9e -324 to 1.8e +308

Reference Types

- Data types that reference objects
- Reference data types are not built into the Java language like primitives
- Can be provided as part of the Java SDK or written by the programmer
- They store data and functionality associated with the data



Primitive Wrappers

Reference Types that add functionality to existing primitives

```
// Byte, Short, Integer, Long, Float, Double, Character, Boolean

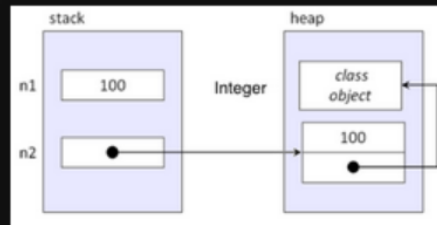
// Primitive type
int primitiveInt = 29;

// wraps the primitive types inside objects
// new is used to create a new object, similar to {} in javascript
Integer objectInt = new Integer("29");

// the wrapper classes provide added functionality
System.out.println("To Binary: " + Integer.toBinaryString(primitiveInt)); // "11101"
System.out.println("To String: " + objectInt.toString()); // "29"
```

Primitives vs Wrappers

```
int n1 = 100;  
Integer n2 = new Integer(100);
```



When should we use it?

- As an argument of methods that expect an object;
- To use object methods (e.g. convert from and to String).
- All objects contain a reference to a corresponding class object, we will have a look at what this is later on

Operators

Assignment and Arithmetic Operators

- Assignment operator is used to assign a value to a variable
- Arithmetic operators are used to perform simple arithmetic operations on primitive data types

```
// assignment
int score = 200; // 200

// arithmetic operators
score = 100 + 5;      // 105
score = 100 - 5;      // 95
score = 100 * 5;       // 500
score = 100 / 5;       // 20
score = 100 % 3;       // 1 - modulus operator (%): (100 = 33 * 3 + 1)
```

Unary and Ternary Operators

- Unary operators need only one operand
- Ternary operators have three operands
- Ternary operators are commonly used to shorthand if-else statements

```
int score = 100;

// pre and post increment/decrement
int highScore = score++; // highScore = 100 and score = 101
highScore = score--; // highScore = 101 and score = 100
highScore = ++score; // highScore = 101 and score = 101
highScore = --score; // highScore = 100 and score = 100
// easy to insert Off-by-one type of bugs if not used on their own

score += 10; // 110 - a shorter version of score = score + 10
score -= 10; // 100
score *= 10; // 1000
score /= 10; // 100

// Usage of the ternary inside expression to replace if/else statement
int x = 10;
System.out.println((x % 2 == 0) ? "yes " + x : "no " + x);

x = 11;
System.out.println((x % 2 == 0) ? "yes " + x : "no " + x);
```

Relational Operators

- Used to check for relations between values
- Evaluate to a boolean value

```
score > 99      // greater than
score < 56      // less than
score >= 76     // greater than or equal to
score <= 83     // less than or equal to
score == 45     // equals
score != 23     // not equal to
```

Logical Operators

- Used for testing multiple conditions
- Evaluate to a boolean result

```
score > 99 && score < 150 // AND, returns true if all are true  
score > 99 || score < 45 // OR, returns true if at least one is true
```

Some Basic Math Methods I

```
// absolute value
int i = -1;
System.out.println(Math.abs(i));

// round number
float f = 10.23f;

int ceil = (int) Math.ceil(f); // returns double
int floor = (int) Math.floor(f); // returns double
int round = Math.round(f); // returns int

System.out.println(ceil + " " + floor + " " + round);
```

Some Basic Math Methods II

```
// min and max
int number1 = 10;
int number2 = 15;

int min = Math.min(number1, number2);
int max = Math.max(number1, number2);

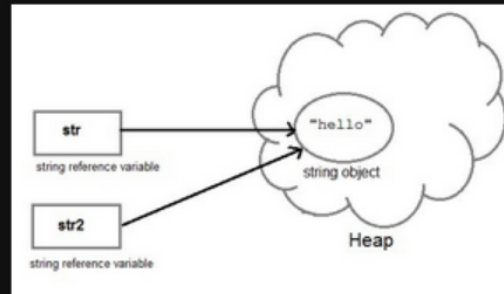
System.out.println(min + " " + max);
```

Strings

A String consists of a sequence of characters



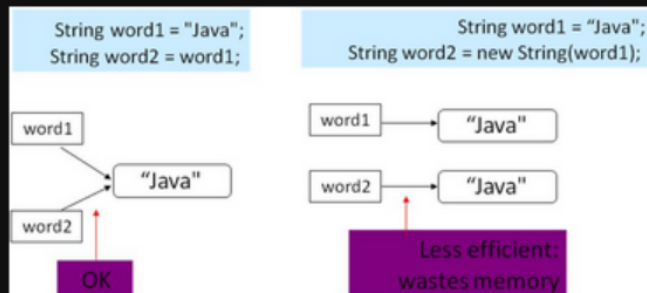
A String in Java is an Object, we access it using a string reference:



Creating Strings

```
// creating a String literal  
String hello = "Hello World";  
  
// creating a String using the constructor  
String world = new String("Hello World");
```

Which one should we use?



Go with String Literal

```
// using literals
String a = "Java";
String b = "Java";

System.out.println(a == b); // True, it is the same object, GO WITH THIS ONE

// using the constructor
String c = new String("Java");
String d = new String("Java");

System.out.println(c == d); // False, they are different objects
```

Compare the String content

```
// using literals
String a = "Java";
String b = "Java";

System.out.println(a.equals(b)); // True, they have the same content

// using the constructor
String c = new String("Java");
String d = new String("Java");

System.out.println(c.equals(d)); // True, they have the same content
```

Concatenating Strings

```
String hello = "Hello ";  
String world = "World!";  
  
System.out.println(hello + world);  
System.out.println(hello.concat(world));  
System.out.println("hello".concat(world));
```

Converting Numbers to String

```
int age = 21;  
  
String str1 = age + "";  
String str2 = String.valueOf(age);  
String str3 = Integer.toString(age);
```

Getting a Character by Index

```
String str = "Hello World!";  
  
// returns the character at the specified index  
char ch = str.charAt(6);  
  
System.out.println(ch);
```

Substring

```
String str = "Hello World!";  
  
// returns the "Hello" substring  
String hello = str.substring(0, 5);  
  
// returns the "World!" substring  
String world = str.substring(6);  
  
System.out.println(hello + " " + world);
```

String Length

```
String str = "Hello World!";  
System.out.println(str.length());
```

{ Exercise }

String: Missing Char

Given a non-empty string and an int n, print a new string where the char at index n has been removed.

```
public static void main(String[] args) {  
    missingChar("Robocop", 3);  
  
    //receive command line argument if available  
    if(args.length>=2){  
        missingChar(args[0], Integer.parseInt(args[1]));  
    }  
}  
  
private static void missingChar(String str, int n) {  
    // print result here  
}
```

{ Exercise }

String: Not String

Given a string, print a new string where "not " has been added to the front. However, if the string already begins with "not", print the string unchanged.

```
public static void main(String[] args) {  
  
    notString("semicolon");  
    notString("not semicolon");  
  
    //receive command line argument if available  
    if(args.length>0){  
        notString(args[0]);  
    }  
}  
  
private static void notString(String str) {  
  
    // print result here  
  
}
```

{ Exercise }

String: Front Back

Given a string, print a new string where the first and last characters have been exchanged.

```
public static void main(String[] args) {  
    frontBack("heisenberg");  
  
    //receive command line argument if available  
    if(args.length>0){  
        frontBack(args[0]);  
    }  
}  
  
private static void frontBack(String str) {  
    // print result here  
}
```

Searching inside a String

```
String str = "Hello World!";

// what to search for, it can be a char or string
String search = "o";

// check if the string contains the search
if(str.contains(search)) {
    System.out.println(search + " exists!");
}

// search forward from the beginning or a specified index
int index = str.indexOf(search); // ??

// search backwards from the end or a specified index
int lastIndex = str.lastIndexOf(search, str.length() - 2); // ??

// if the string does not contain the search expression, -1 is returned

System.out.println("The first " + search + " is at " + index +
    " and the last one at " + lastIndex);
```

Replacing String content

```
String str = "Hello World!";

// what to replace and for what
String target = "Hello";
String replacement = "Goodbye";

// create a new string with the provided replacement (strings are immutable)
str = str.replace(target, replacement);

// replace only the first occurrence
str = str.replaceFirst("o", "U");

System.out.println(str);
```

Searching and Replacing with Regular Expressions

```
// Replace all digits with letter X
str = str.replaceAll("[0-9]", "X");

// Replace all occurrences of multiple spaces with one single space
str = str.replaceAll(" {2,}", " ");
```

Splitting with Regular Expressions

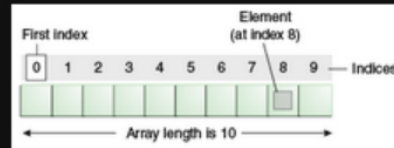
```
String str = "Academia deCodigo";  
  
String[] words = str.split(" "); // str.split(" ", 2) to limit the array length  
  
// words = ["Academia", "de", "Codigo"]  
// prints the first element of the array  
System.out.println(words[0]);
```

{ Exercise }

String Manipulations

```
String str = "http://www.academiadecodigo.org";  
  
String domain = ""; // www.academiadecodigo.org  
String name = "";   // < Academia deCodigo_ >  
  
// print the following message at the end  
// I am a Code Cadet at < Academia deCodigo_ >, www.academiadecodigo.org
```

Arrays



An array is a container object that holds a fixed number of values of a single type

Declaring an Array

```
// The length of an array is established when the array is created.  
// After creation, its length is fixed.  
String[] contacts = new String[4];  
  
// declare an array with values  
String[] contacts = { "John", "Mary", "Ze" };
```

Accessing an Array

```
String[] contacts = new String[4];  
  
// assign a value to one element of the array  
contacts[0] = "Ze";  
  
// retrieve the value of one element  
String ze = contacts[0];
```

Array Length

```
// declare an array with values  
String[] contacts = { "John", "Mary", "Ze" };  
  
// print the length  
System.out.println(contacts.length);
```

{ Exercise }

Closest Neighbours

Find the two neighbouring numbers in an array with the smallest difference to each other

```
public static void main(String[] args) {  
    int[] myArray = {0, 5, 1209, 6, 2, 111, 112, 33};  
    int[] result = findClosest(myArray);  
    // print the elements of the resulting array  
}  
  
private static int[] findClosest(int[] numbers) {  
    // hint: remember Math.abs();  
}
```

Control Flow Statements

Control flow statements, break up the flow of execution by employing decision making, looping, and branching, enabling your program to conditionally execute particular blocks of code.

The if-then Statement

```
int bullets = 7;  
if(bullets > 0) {  
    System.out.println("Shoot");  
}
```

The if-then-else Statement

```
int bullets = 7;  
if(bullets > 0) {  
    System.out.println("Shoot");  
} else {  
    System.out.println("Reload");  
}
```

The if-then-else Statement

Multiple Conditions

```
// score
int score = 95;

// player ranking
if(score > 0) {

    System.out.println("NOOB!");

} else if (score > 30) {

    System.out.println("Not bad...");

} else if (score > 60) {

    System.out.println("That was good.");

} else if (score > 90) {

    System.out.println("Awesome!");

}
```

This is the wrong way to do it!

The if-then-else Statement

The order matters!

```
// score
int score = 95;

// player ranking
if(score > 90) {

    System.out.println("Awesome!");
} else if (score > 60) {

    System.out.println("That was good");
} else if (score > 30) {

    System.out.println("Not bad...");
} else {

    System.out.println("NOOB!");
}
```

The Switch Statement

```
// weapons: 0 - Sword 1 - Gun 2 - Grenade
int currentWeapon = 0;

switch(currentWeapon) {

    case 0:
        System.out.println("Attack!");
        break;

    case 1:
        System.out.println("Shoot!");
        break;

    case 2:
        System.out.println("Throw!");
        break;

    default:
        System.out.println("Get a weapon...");
        break;
}
```

The For Loop

```
String[] contacts = { "John", "Mary", "Jean", "Paul" };  
  
for(int i = 0; i < contacts.length; i++) {  
    // print each contact  
    System.out.println(contacts[i]);  
}
```

The Enhanced For Loop

```
String[] contacts = { "John", "Mary", "Jean", "Paul"};

for (String contact : contacts) {

    // print each contact
    System.out.println(contact);
}
```

The While Loop

```
// bullet count
int bullets = 100;

while (bullets > 0) {

    // shoot the weapon
    bullets--;

}
```

Be aware of infinite loops!

Breaking from a Switch

```
// weapons: 0 - Sword 1 - Gun 2 - Grenade
int currentWeapon = 0;

switch(currentWeapon) {

    case 0:
        System.out.println("Attack!");
        break;

    case 1:
        System.out.println("Gun!");
        break;

    case 2:
        System.out.println("Throw!");
        break;

    default:
        System.out.println("Get a weapon...");
        break;
}
```

Breaking from a Loop

```
String[] droids = { "M-TD", "C3P0", "R2-D2", "RA-7" };

// we are looking for R2-D2
for(int i = 0; i < droids.length; i++) {

    if(droids[i].equals("R2-D2")) {

        // do something with it and jump out of the loop
        System.out.println("This is the droid I was looking for...");

        break;

    }

    System.out.println("Still looking...");
}
```

The Continue

```
String[] droids = { "M-TD", "C3P0", "R2-D2", "RA-7" };

// looking for R2-D2 and C3P0
for(int i = 0; i < droids.length; i++) {

    if(!droids[i].equals("R2-D2") && !droids[i].equals("C3P0")) {

        // just skip the iteration
        System.out.println("This is not the droid I'm looking for...");

        continue;

    }

    // do something with the droid
    System.out.println("Found " + droids[i]);

}
```